

Evaluación del rendimiento computacional de modelos de machine learning para detección de ciberataques en entornos local y cloud con estrategias de optimización

Aiko Masaki Perales
Universidad del Pacífico
Lima, Perú

Mildred Bautista Abanto
Universidad del Pacífico
Lima, Perú

Jean Trujillo Dominguez
Universidad del Pacífico
Lima, Perú

Sebastian Minchola Robles
Universidad del Pacífico
Lima, Perú

AV.MasakiP@alum.up.edu.pe MB.BautistaA@alum.up.edu.pe JL.TrujilloD@alum.up.edu.pe WS.MincholaR@alum.up.edu.pe

Resumen—La detección eficiente de ciberataques en redes de gran escala requiere no solo modelos predictivos precisos, sino también infraestructuras computacionales capaces de procesarlos en tiempos razonables. Este trabajo evalúa el impacto de estrategias de optimización en el rendimiento de modelos de machine learning (Random Forest, XGBoost y MLP) aplicados al dataset UNSW-NB15 (700K registros), comparando un pipeline sin optimización frente a uno optimizado en entorno local. Los resultados muestran reducciones de hasta 83 % en tiempo de entrenamiento para modelos basados en ensamble sin degradación significativa de F1-score. Como trabajo futuro se proyecta extender los experimentos a una VM en Microsoft Azure para comparar el rendimiento entre entornos on-premise y cloud.

Index Terms—Machine Learning, Ciberseguridad, Cloud Computing, HPC, Optimización

I. INTRODUCCIÓN

En los últimos años, el crecimiento exponencial del volumen de datos y la sofisticación de los ciberataques han impulsado la necesidad de desarrollar soluciones eficientes para la detección de intrusiones en redes. Según reportes recientes de organismos como el National Institute of Standards and Technology (NIST), la ciberseguridad se ha convertido en un componente crítico en la infraestructura tecnológica moderna, donde la capacidad de procesar grandes volúmenes de datos en tiempos reducidos es fundamental para la toma de decisiones en tiempo real (National Institute of Standards and Technology, 2023). Asimismo, estudios de la industria como IBM Security Report destacan que el tiempo de respuesta ante amenazas está directamente relacionado con la capacidad computacional disponible y la eficiencia de los algoritmos utilizados (IBM Security, 2023).

En este contexto, la Computación en la Nube (Cloud Computing) y la Computación de Alto Desempeño (High Performance Computing, HPC) han emergido como alternativas clave para abordar problemas de gran escala, como el entrenamiento de modelos de aprendizaje automático para la detección de ciberataques. Sin embargo, aún existe incertidumbre respecto a cuándo resulta más conveniente utilizar infraestructura cloud frente a entornos locales (on-premise),

especialmente considerando factores como el costo, la escalabilidad y el rendimiento computacional.

A partir de esta problemática, surge la necesidad de analizar comparativamente el desempeño de diferentes infraestructuras en tareas de machine learning aplicadas a ciberseguridad, considerando no solo la precisión de los modelos, sino también el tiempo de procesamiento y el uso de recursos computacionales. En particular, este trabajo se enfoca en evaluar el impacto del uso de entornos cloud frente a entornos locales, así como el efecto de técnicas de optimización en el rendimiento del entrenamiento de modelos.

En este sentido, se plantean las siguientes preguntas de investigación:

- ¿Qué diferencias existen en el rendimiento computacional entre una infraestructura local y una infraestructura cloud al entrenar modelos de detección de ciberataques?
- ¿En qué medida las técnicas de optimización influyen en la reducción del tiempo de entrenamiento y uso de recursos?
- ¿Qué combinación de infraestructura y optimización ofrece la mejor relación entre rendimiento, costo y precisión del modelo?

El objetivo general de este trabajo es evaluar y comparar el rendimiento, costo y escalabilidad del entrenamiento de modelos de machine learning para la detección de ciberataques en entornos on-premise y cloud.

Como objetivos específicos, se plantean:

- Implementar modelos de clasificación (Random Forest, XGBoost y Redes Neuronales) aplicados a un dataset de ciberseguridad.
- Medir y analizar el tiempo de ejecución en distintas etapas del pipeline (carga de datos, preprocesamiento, entrenamiento e inferencia).
- Evaluar el uso de recursos computacionales (CPU) en ambos entornos.
- Comparar el desempeño predictivo de los modelos mediante métricas como accuracy, precision, recall y F1-

score.

- Analizar el costo asociado al uso de infraestructura cloud en comparación con el entorno local.
- Evaluar el impacto de técnicas de optimización en el rendimiento del sistema.

Finalmente, se plantean las siguientes hipótesis de investigación:

- H1: El uso de infraestructura cloud permitirá reducir el tiempo de entrenamiento en comparación con un entorno local, especialmente para datasets de gran tamaño.
- H2: La aplicación de técnicas de optimización (paralelización y mejoras en el procesamiento de datos) reducirá significativamente el tiempo de ejecución en ambos entornos.
- H3: Los modelos basados en técnicas de ensamble, como XGBoost, presentarán una mejor relación entre rendimiento predictivo y tiempo de entrenamiento en comparación con otros modelos evaluados.
- H4: El uso de infraestructura cloud implicará un mayor costo económico, pero ofrecerá ventajas en términos de escalabilidad y flexibilidad.

II. TRABAJOS RELACIONADOS

En los últimos años, la detección de ciberataques mediante técnicas de machine learning ha evolucionado significativamente, incorporando no solo modelos más sofisticados, sino también nuevas consideraciones relacionadas con el rendimiento computacional, el consumo de recursos y la escalabilidad de los sistemas. A continuación, se presentan tres trabajos recientes que abordan estos aspectos desde diferentes perspectivas.

El estudio de (Ali y cols., 2024) presenta una revisión actualizada de estrategias de machine learning aplicadas a sistemas de detección de intrusiones (IDS). Los autores analizan diversos algoritmos, métricas de evaluación y datasets utilizados en el área, destacando la importancia de modelos como Random Forest, XGBoost y redes neuronales en la detección de tráfico malicioso. Asimismo, el trabajo resalta limitaciones actuales relacionadas con la escalabilidad y la eficiencia computacional de estos enfoques. Sin embargo, al tratarse de una revisión, no se realiza una evaluación experimental comparativa entre distintas infraestructuras de ejecución, lo que deja abierta la necesidad de estudios empíricos en este ámbito.

Por otro lado, (Ajagbe y cols., 2024) realizan un estudio comparativo de modelos de machine learning aplicados al dataset UNSW-NB15, uno de los más utilizados en la detección de intrusiones. En este trabajo se evalúan algoritmos como Random Forest y XGBoost utilizando métricas como accuracy, precision, recall y F1-score, demostrando que los métodos de ensamble presentan un desempeño superior frente a otros modelos. Este estudio resulta relevante para la selección de algoritmos en el presente trabajo; sin embargo, su enfoque se centra principalmente en la calidad predictiva, sin considerar aspectos relacionados con el tiempo de entrenamiento, el uso de recursos computacionales o el impacto de la infraestructura utilizada.

Finalmente, el trabajo de (Jamshidi y cols., 2025) analiza el desempeño de sistemas de detección de intrusiones basados en machine learning en entornos IoT, considerando no solo la precisión de los modelos, sino también métricas de rendimiento como el uso de CPU, carga del sistema y consumo energético. Los resultados evidencian que el despliegue de estos sistemas puede generar un impacto significativo en los recursos computacionales, especialmente en escenarios de alta demanda. No obstante, el estudio se enfoca en entornos edge/IoT y no aborda comparaciones directas entre infraestructuras cloud y on-premise, ni el impacto de técnicas de optimización en el proceso de entrenamiento.

En conjunto, estos trabajos muestran que, si bien existe un amplio avance en el desarrollo de modelos de detección de intrusiones y en la evaluación de su desempeño, persiste una brecha en el análisis integral que combine la calidad predictiva, la eficiencia computacional y la comparación entre diferentes infraestructuras de ejecución. En este contexto, el presente trabajo busca contribuir mediante un enfoque experimental que integre estos elementos, evaluando el impacto del uso de entornos cloud frente a entornos locales, así como la aplicación de técnicas de optimización en el rendimiento del entrenamiento de modelos de machine learning.

III. METODOLOGÍA

III-A. Formulación del problema

El presente trabajo aborda la detección de ciberataques como un problema de clasificación binaria, donde cada instancia del tráfico de red es clasificada como normal o maliciosa. A diferencia de enfoques tradicionales, se incorpora una perspectiva de computación de alto desempeño (HPC), en la cual el objetivo no solo es maximizar la calidad predictiva, sino también optimizar el rendimiento computacional del sistema.

III-B. Descripción del dataset

Se utilizó el dataset UNSW-NB15 (Moustafa y Slay, 2015) con aproximadamente 700,000 registros de tráfico de red. Cada observación está descrita mediante 49 variables: 43 numéricas y 6 categóricas.

La variable objetivo es *label*, mientras que *attack_cat* describe el tipo específico de ataque y se utilizó únicamente para análisis exploratorio.

III-C. Análisis exploratorio de datos

Se realizó un análisis exploratorio para comprender la estructura del dataset. La Tabla I muestra la distribución de la variable objetivo, evidenciando un fuerte desbalance de clases.

Cuadro I
DISTRIBUCIÓN DE LA VARIABLE OBJETIVO

Clase	Cantidad	Porcentaje
Normal (0)	677786	96.83 %
Ataque (1)	22215	3.17 %

Este desbalance implica la necesidad de utilizar métricas adicionales como F1-score, además de accuracy.

La Figura 1 muestra visualmente esta distribución.

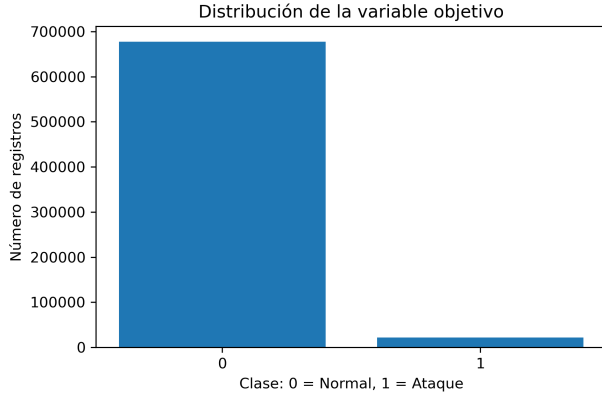


Figura 1. Distribución de la variable objetivo

Asimismo, la Figura 2 presenta la distribución de las categorías de ataque.

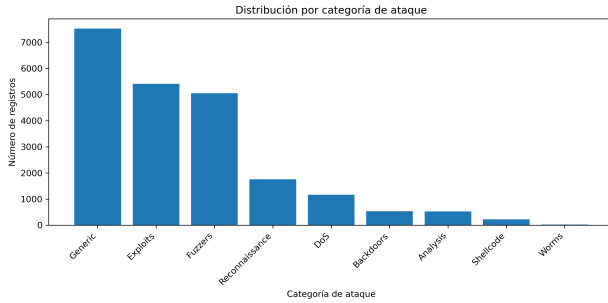


Figura 2. Distribución por categoría de ataque

El análisis de correlación (Figura 3) permitió identificar variables con alta relación respecto a la variable objetivo.

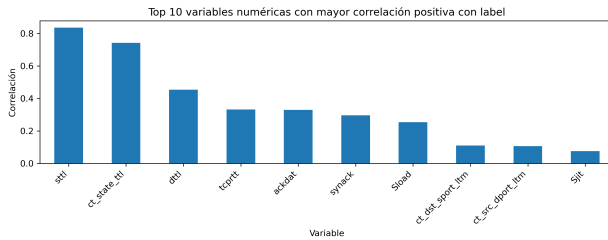


Figura 3. Variables con mayor correlación con la variable objetivo

III-D. Preprocesamiento de datos

A partir del análisis exploratorio, se aplicaron las siguientes transformaciones:

- Eliminación de las variables categóricas *srcip* y *dstip*, debido a su alta cardinalidad.
- Exclusión de *attack_cat* para evitar fuga de información.
- Conversión de *sport* y *dport* a formato numérico.
- Imputación de valores faltantes.
- Codificación de variables categóricas mediante Label Encoding.

Adicionalmente, se aplicó una optimización de memoria orientada a reducir el consumo de RAM durante el procesamiento del dataset. Esta optimización consistió en ajustar los tipos de datos numéricos de acuerdo con el rango de valores de cada variable. En particular, las columnas enteras fueron convertidas desde tipos de mayor tamaño, como *int64*, hacia tipos más livianos, como *int8*, *int16* o *int32*, según correspondiera. De forma similar, las variables flotantes fueron convertidas de *float64* a *float32* cuando no implicaba pérdida relevante de precisión para el entrenamiento.

Esta estrategia permitió disminuir el espacio ocupado en memoria, reducir la carga computacional asociada al procesamiento de datos y mejorar la eficiencia del pipeline antes del entrenamiento de los modelos. La Tabla II muestra la reducción obtenida.

Cuadro II
OPTIMIZACIÓN DE MEMORIA

Memoria antes (MB)	Memoria después (MB)	Reducción (%)
245.67	92.13	62.50 %

III-E. Pipeline de procesamiento

El pipeline de procesamiento implementado sigue una secuencia estructurada que incluye carga de datos, análisis exploratorio, preprocesamiento, división del dataset, entrenamiento y evaluación de modelos. No obstante, a diferencia de un enfoque tradicional, este trabajo introduce dos variantes del pipeline con el objetivo de evaluar el impacto de las estrategias de optimización en el rendimiento computacional.

III-F. Diseño experimental

El experimento se estructura en dos escenarios principales en entorno local, como se muestra en la Figura 4:

- **Escenario 1 (Local sin optimización):** pipeline ejecutado sin aplicar optimizaciones ni en el procesamiento de datos ni en los modelos.
- **Escenario 3 (Local optimizado):** pipeline completo con optimizaciones tanto en el preprocesamiento como en el entrenamiento de modelos.

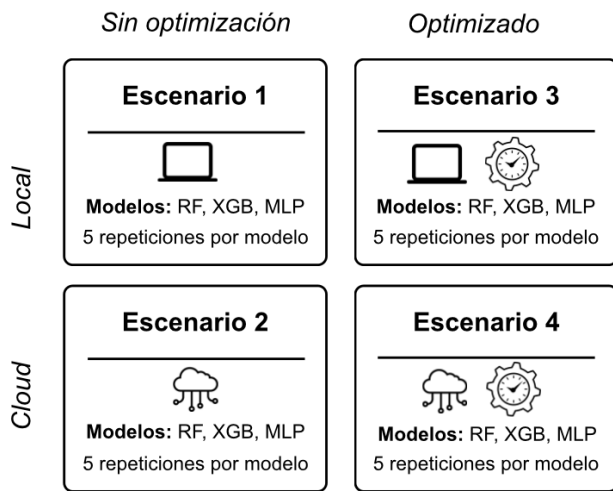


Figura 4. Diseño experimental de escenarios evaluados

Este enfoque permite comparar de manera directa el impacto de la optimización a nivel de pipeline completo, en lugar de evaluar únicamente configuraciones aisladas de modelos.

III-G. Definición de pipelines

Se definieron dos pipelines diferenciados:

Pipeline sin optimización:

- Transformaciones mínimas necesarias para el entrenamiento.
- Codificación de variables categóricas.
- Imputación básica de valores faltantes.
- Sin reducción de memoria.
- Sin ajuste de tipos de datos.
- Modelos ejecutados en configuración secuencial ($n_jobs=1$).

Pipeline optimizado:

- Eliminación de variables de alta cardinalidad (*srcip*, *dstip*).
- Exclusión de *attack_cat*.
- Conversión eficiente de variables de puertos a formato numérico.
- Imputación controlada de valores faltantes.
- Codificación de variables categóricas.
- Reducción de memoria mediante ajuste de tipos de datos.
- Modelos ejecutados en paralelo ($n_jobs=-1$).
- Uso de configuraciones optimizadas en XGBoost.

III-H. Modelos de clasificación

Se utilizaron tres modelos de machine learning:

- Random Forest
- XGBoost
- MLPClassifier

En el pipeline optimizado, Random Forest se configuró con *class_weight='balanced'* para mitigar el desbalance de clases.

III-I. Infraestructura experimental

Los experimentos locales (Escenarios 1 y 3) fueron ejecutados en un equipo con procesador Intel Core i7 y 32 GB de RAM. Para los Escenarios 2 y 4, se proyecta replicar el mismo pipeline en una máquina virtual en Microsoft Azure, configurada con especificaciones equivalentes de cómputo, con el objetivo de aislar el efecto de la infraestructura como única variable diferenciadora respecto a los escenarios locales.

III-II. Arquitectura de paralelización (Escenarios 3 y 4):

La Figura 5 ilustra la arquitectura del pipeline optimizado. La etapa de carga y preprocesamiento opera de forma secuencial, dado que depende de operaciones de I/O sobre el dataset completo. A partir de la etapa de entrenamiento, cada modelo es ejecutado con paralelización interna habilitada mediante $n_jobs=-1$, lo que permite distribuir los árboles de decisión (Random Forest) o los *boosting rounds* (XGBoost) entre los núcleos disponibles del procesador. La capa de evaluación agrega los resultados en una tabla comparativa unificada. Esta misma arquitectura será replicada en el Escenario 4 sobre la VM en Azure.

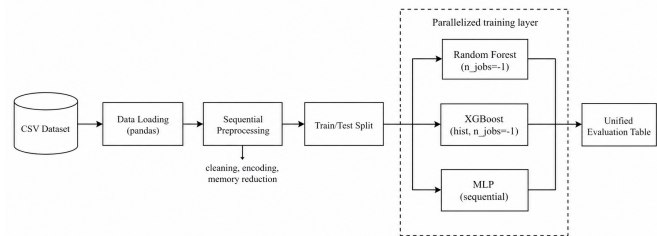


Figura 5. Arquitectura del pipeline optimizado (Escenarios 3 y 4)

III-I2. Arquitectura cloud (Escenarios 2 y 4 — Azure): La Figura 6 presenta la arquitectura proyectada para el entorno cloud. Se planea desplegar una VM en Microsoft Azure (serie *Dsv3* o equivalente), replicando los mismos componentes de software del entorno local: misma versión de Python, mismas librerías y mismo dataset. El acceso se realizará vía SSH desde la máquina local, y los resultados serán descargados para análisis comparativo. El Escenario 2 ejecutará el pipeline sin optimizaciones, mientras que el Escenario 4 incorporará las mismas estrategias del Escenario 3 (paralelización, reducción de memoria y configuración eficiente de XGBoost), permitiendo así una comparación cruzada de los efectos de infraestructura y optimización de forma independiente.

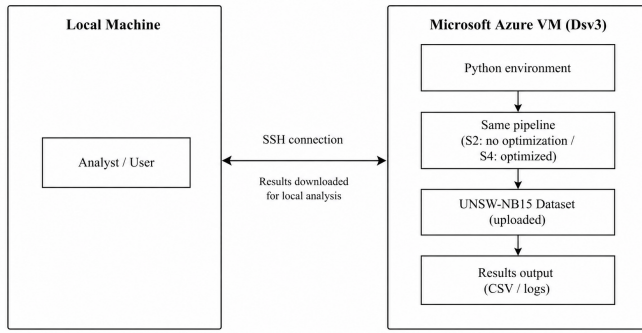


Figura 6. Arquitectura de los Escenarios 2 y 4 en Microsoft Azure

III-J. Resultados computacionales

La evaluación del rendimiento se realizó considerando métricas tanto predictivas como computacionales, permitiendo analizar el impacto de la optimización a nivel de pipeline completo.

La Figura 7 presenta el tiempo de entrenamiento por modelo en ambos escenarios. Se observa una reducción significativa en los tiempos para Random Forest y XGBoost en el escenario optimizado, mientras que el modelo MLP no presenta mejoras, evidenciando que no todos los algoritmos se benefician de la misma forma de las optimizaciones aplicadas.

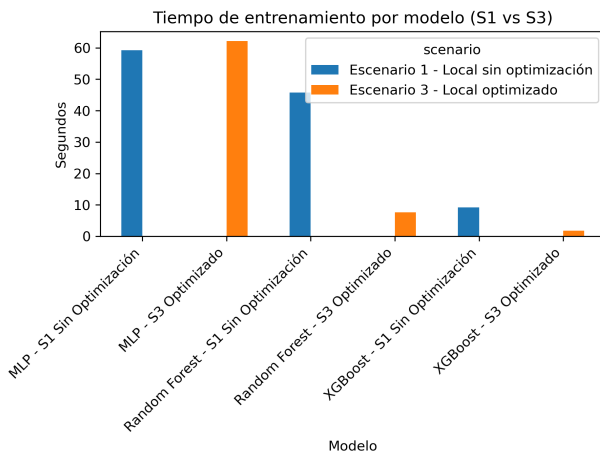


Figura 7. Tiempo de entrenamiento por modelo (Escenario 1 vs Escenario 3)

De manera complementaria, la Figura 8 muestra el tiempo de inferencia, donde también se observan reducciones importantes en los modelos basados en ensemble.

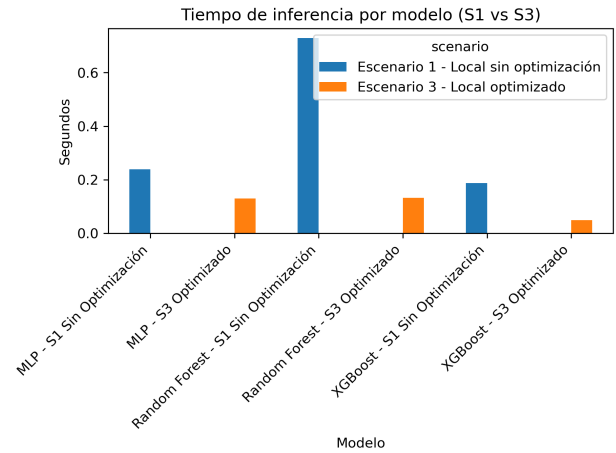


Figura 8. Tiempo de inferencia por modelo (Escenario 1 vs Escenario 3)

La Figura 9 presenta el desempeño en términos de F1-score. Los resultados indican que la optimización del pipeline no genera degradación significativa en la calidad predictiva de los modelos.

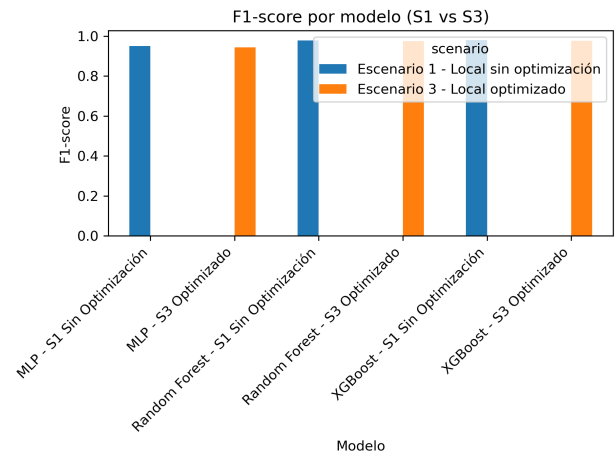


Figura 9. F1-score por modelo (Escenario 1 vs Escenario 3)

La Tabla III resume cuantitativamente la reducción en tiempos de entrenamiento e inferencia.

Cuadro III
COMPARACIÓN DE RENDIMIENTO ENTRE ESCENARIOS

Modelo	Reducción entrenamiento (%)	Reducción inferencia (%)	Variación F1-score (%)
Random Forest	83.40 %	81.83 %	-0.00 %
XGBoost	80.51 %	73.95 %	-0.00 %
MLP	-4.95 %	45.75 %	-0.00 %

Los resultados evidencian que las estrategias de optimización aplicadas permiten reducir de manera sustancial los tiempos de entrenamiento e inferencia en modelos basados en árboles, particularmente Random Forest y XGBoost, sin afectar de forma significativa su desempeño predictivo.

Por otro lado, el modelo MLP presenta un comportamiento distinto, mostrando una ligera degradación en tiempo de

entrenamiento y en métricas de desempeño, lo cual sugiere que este tipo de modelos no se beneficia directamente de las optimizaciones consideradas en este estudio.

IV. RESULTADOS EXPERIMENTALES

IV-A. Alcance del informe parcial

En esta entrega parcial se reportan los resultados correspondientes a los Escenarios 1 y 3, ambos ejecutados en entorno local. Los Escenarios 2 (cloud sin optimización) y 4 (cloud optimizado) serán implementados sobre una VM en Microsoft Azure y sus resultados serán incorporados en el informe final, completando el análisis comparativo cruzado entre infraestructura y nivel de optimización.

IV-B. Impacto de la optimización del pipeline

Los resultados obtenidos en el entorno local demuestran que las estrategias de optimización —paralelización interna con `n_jobs=-1`, reducción de memoria y ajuste de configuración en XGBoost— generan mejoras sustanciales en el rendimiento computacional sin comprometer la calidad predictiva.

La Tabla IV presenta el resumen completo de métricas predictivas y computacionales para los Escenarios 1 y 3.

Cuadro IV
RESULTADOS COMPLETOS POR MODELO Y ESCENARIO

Escenario	Modelo	T. Entreno (s)	T. Inf. (s)	Acc.	Prec.	Rec.	F1
S1	RF	45.80	0.729	0.9986	0.9684	0.9874	0.9778
S1	XGB	9.19	0.188	0.9987	0.9733	0.9847	0.9790
S1	MLP	59.19	0.239	0.9969	0.9461	0.9552	0.9506
S3	RF	7.60	0.132	0.9983	0.9728	0.9750	0.9739
S3	XGB	1.79	0.049	0.9985	0.9719	0.9800	0.9759
S3	MLP	62.12	0.130	0.9964	0.9428	0.9426	0.9427

Como se aprecia en la Tabla III, Random Forest y XGBoost presentan reducciones en tiempo de entrenamiento del 83.4 % y 80.5 % respectivamente al pasar del Escenario 1 al 3. El modelo MLP no se beneficia de las mismas optimizaciones, dado que su cuello de botella reside en la propagación hacia adelante y hacia atrás, operaciones que no se distribuyen mediante `n_jobs`. Cabe destacar que en ningún caso la optimización generó una degradación significativa en las métricas predictivas, con variaciones de F1-score inferiores a 0.008.

IV-C. Uso de recursos computacionales

La Figura 10 muestra el uso de CPU durante el entrenamiento. Se observa que XGBoost en el Escenario 3 registra el mayor uso de CPU (11.2 %), lo cual es consistente con la habilitación de `tree_method='hist'` y paralelización explícita. Random Forest también eleva su consumo de CPU al distribuir los árboles entre todos los núcleos disponibles. El modelo MLP, al no beneficiarse de `n_jobs`, mantiene un uso de CPU similar en ambos escenarios.

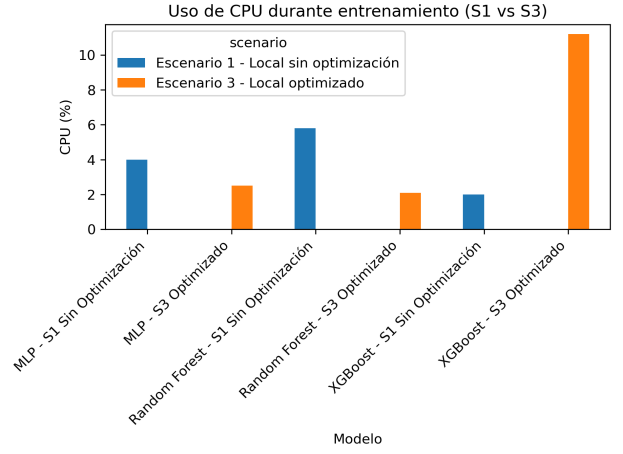


Figura 10. Uso de CPU durante entrenamiento (S1 vs S3)

IV-D. Proyección de escenarios cloud (S2 y S4)

Para el informe final se ejecutarán los Escenarios 2 y 4 sobre la VM en Azure, lo que permitirá responder cuatro preguntas comparativas de forma cruzada: (i) el efecto puro de la infraestructura cloud manteniendo el mismo pipeline sin optimizar (S1 vs S2), (ii) el efecto puro de la optimización en entorno local (S1 vs S3), (iii) el efecto puro de la optimización en entorno cloud (S2 vs S4), y (iv) la combinación óptima de infraestructura y optimización (S3 vs S4). Este diseño experimental permitirá desacoplar los efectos de cada factor y establecer recomendaciones fundamentadas sobre cuándo conviene migrar a cloud y cuándo basta con optimizar el entorno local.

REFERENCIAS

- Ajagbe, S. A., y cols. (2024). Intrusion detection: A comparison study of machine learning models using unbalanced dataset. *SN Computer Science*. Descargado de <https://d-nb.info/1354310276/34> (Accessed: 2026-04-20)
- Ali, M., y cols. (2024). Unveiling machine learning strategies and considerations for intrusion detection systems. *Frontiers in Computer Science*. Descargado de <https://www.frontiersin.org/articles/10.3389/fcomp.2024.1387354/full> (Accessed: 2026-04-20)
- IBM Security. (2023). *Cost of a data breach report 2023*. Descargado de <https://www.ibm.com/reports/data-breach> (Accessed: 2026-04-20)
- Jamshidi, M., y cols. (2025). Evaluating machine learning-driven intrusion detection systems in iot: Performance and energy consumption. *Computer Communications*. Descargado de <https://www.sciencedirect.com/science/article/abs/pii/S0360835225002499> (Accessed: 2026-04-20)
- Moustafa, N., y Slay, J. (2015). UNSW-NB15: A comprehensive data set for network intrusion detection systems. En *2015 military communications and information systems conference (milcis)* (pp. 1–6). doi: 10.1109/MilCIS.2015.7348942
- National Institute of Standards and Technology. (2023). *Cybersecurity framework (csf) 2.0*. Descargado de <https://www.nist.gov/cyberframework> (Accessed: 2026-04-20)